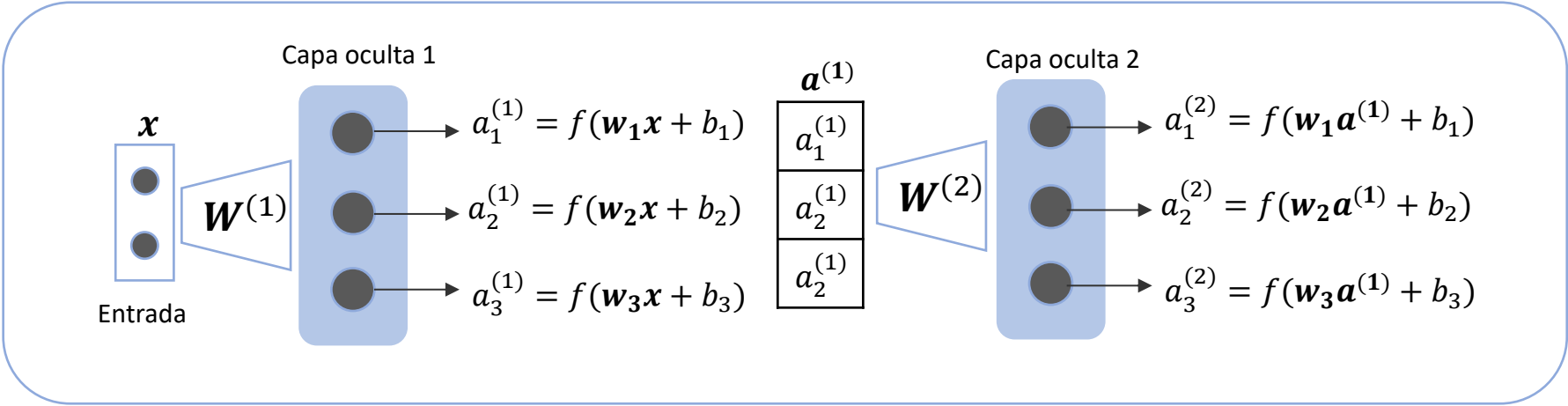


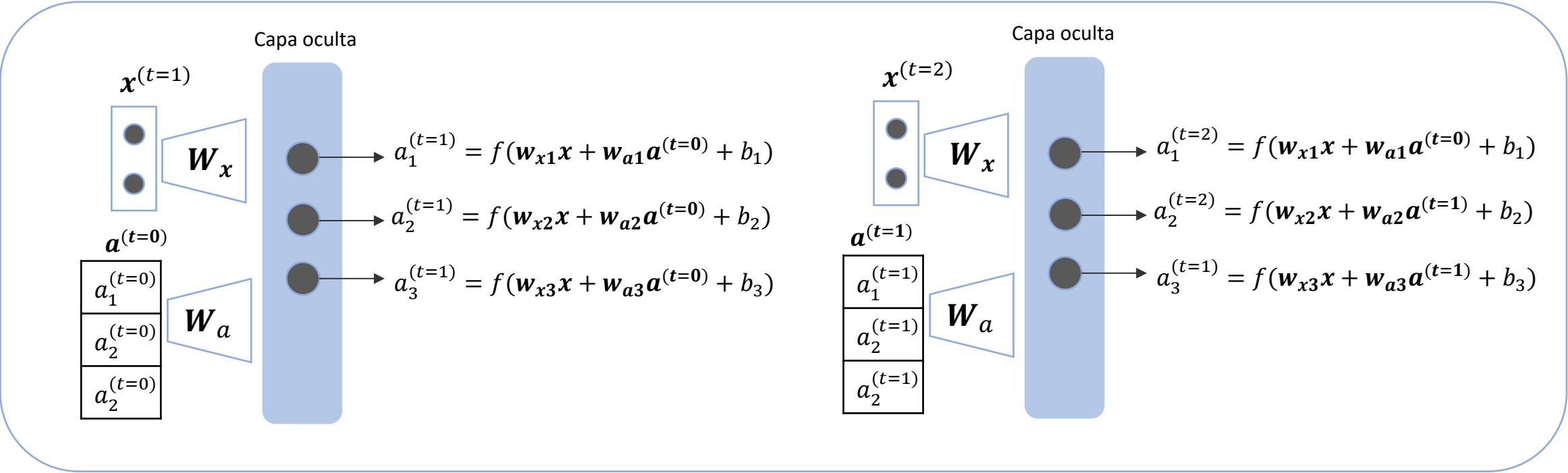
Sesión 13. Mecanismos de atención – El Transformer

- Mecanismos de atención.
- El modelo Transformer.
- Modelos de lenguaje

En MLP:

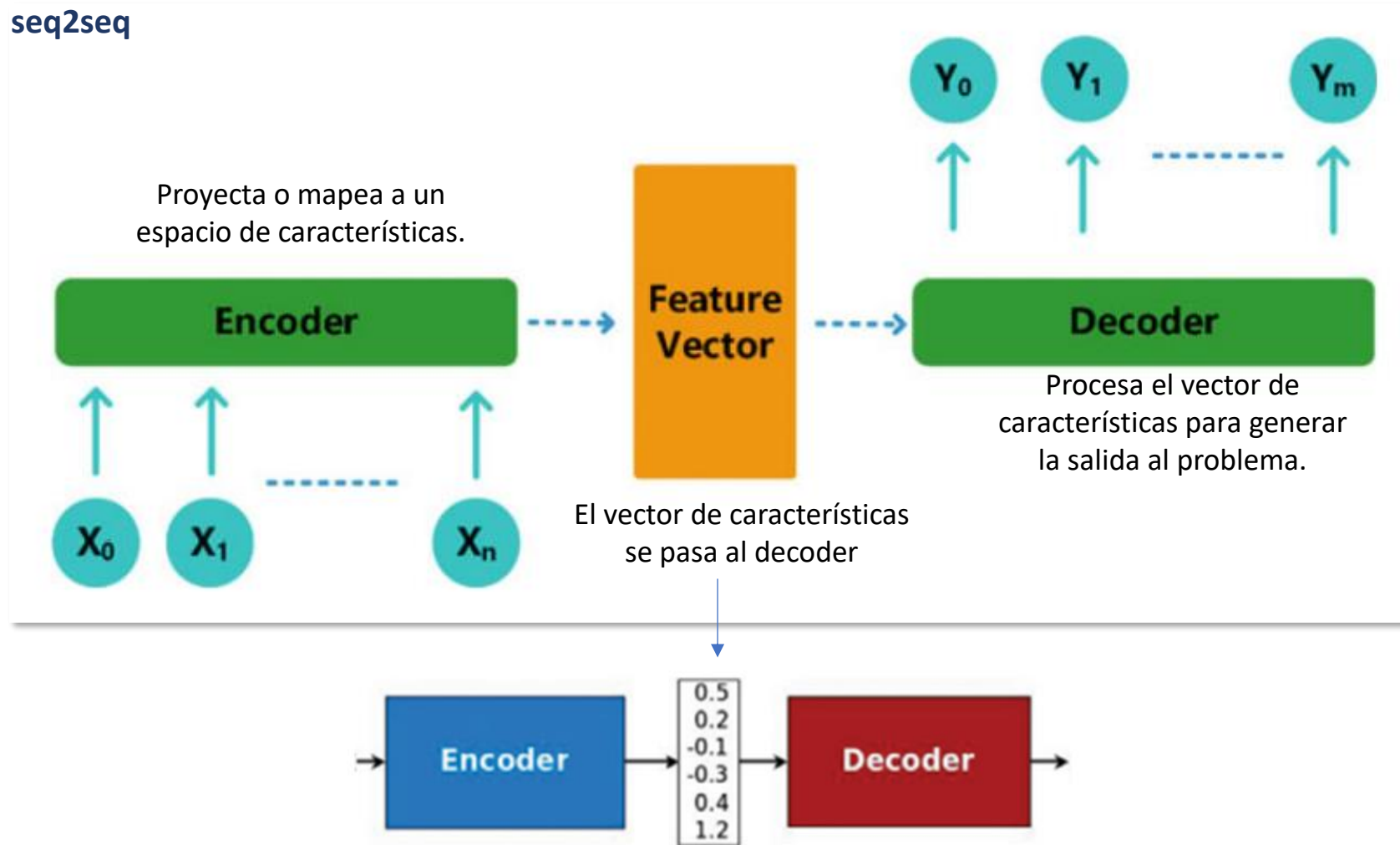


En RNN:

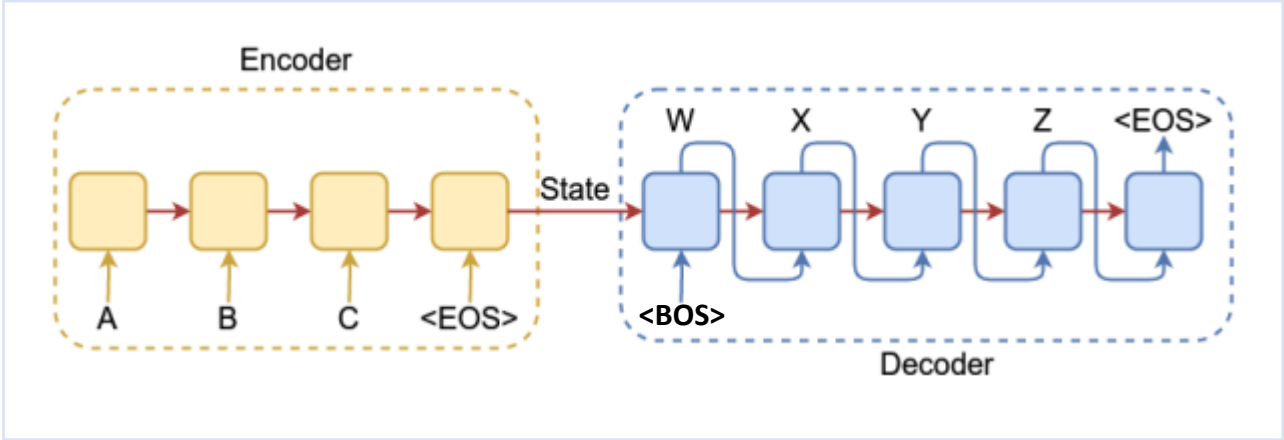


Mecanismo de atención

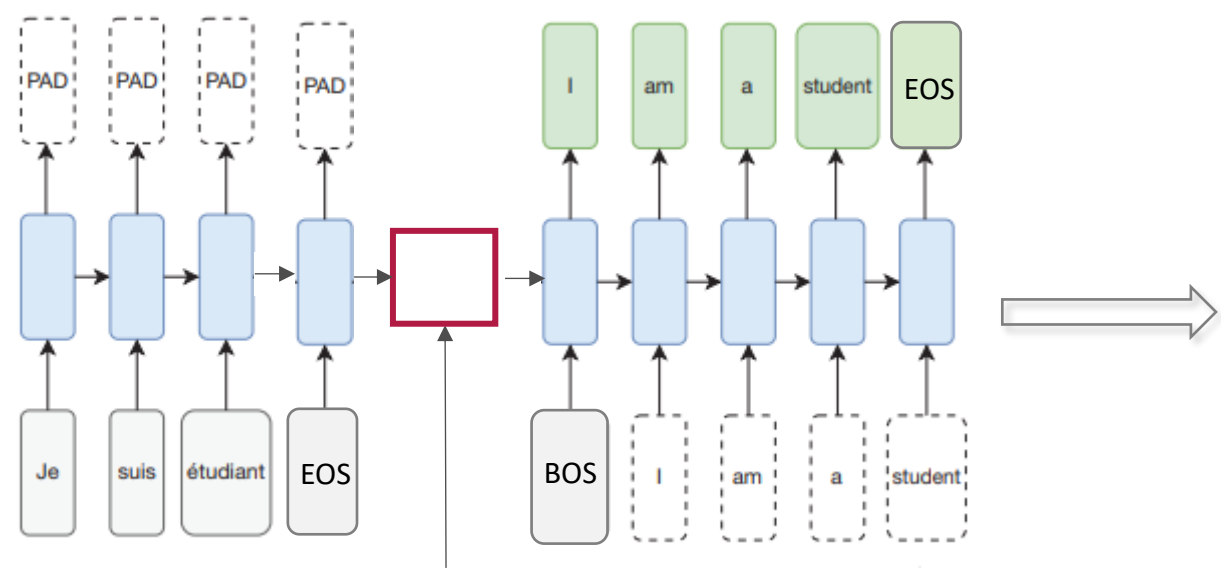
Arquitectura seq2seq



Ejemplo: traslación del lenguaje.

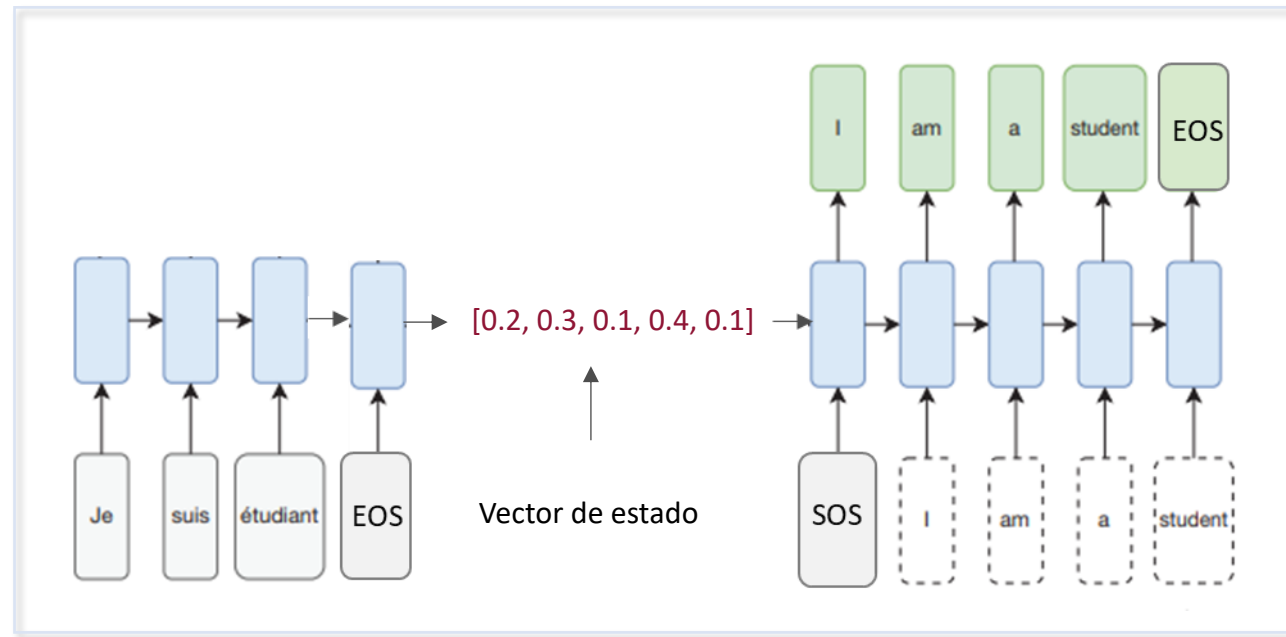


En la práctica se utiliza en el entrenamiento una estrategia de *teacher forcing*.



Vector de características
(vector de estado o latente)
Representa una frase completa

Paso de tiempo	Entrada	Salida
0	Je	PAD
1	suis	PAD
2	étudiant	PAD
3	EOS	PAD
4	SOS	I
5	I	am
6	am	a
7	a	student
8	student	EOS



¿Qué pasa con esta arquitectura?

- El encoder trata de comprimir toda la secuencia de entrada en un vector de tamaño fijo (vector de estado). El tamaño del vector es fijo independiente del tamaño de la secuencia de entrada.
- El decoder trata de reconstruir la secuencia target a partir de este vector.

Idea: incorporar un mecanismo que le permita al decoder acceder a partes específicas de la secuencia de entrada (en el encoder)

Atención: capacidad del cerebro para concentrarse selectivamente en un aspecto del entorno e ignorar otras cosas.



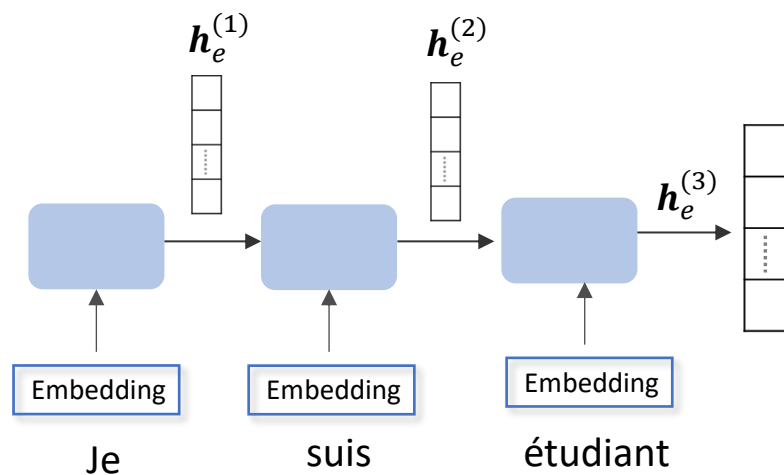
En **machine learning**: mecanismo que le permite a los modelos enfocarse en partes específicas de la entrada.

¿Cómo?

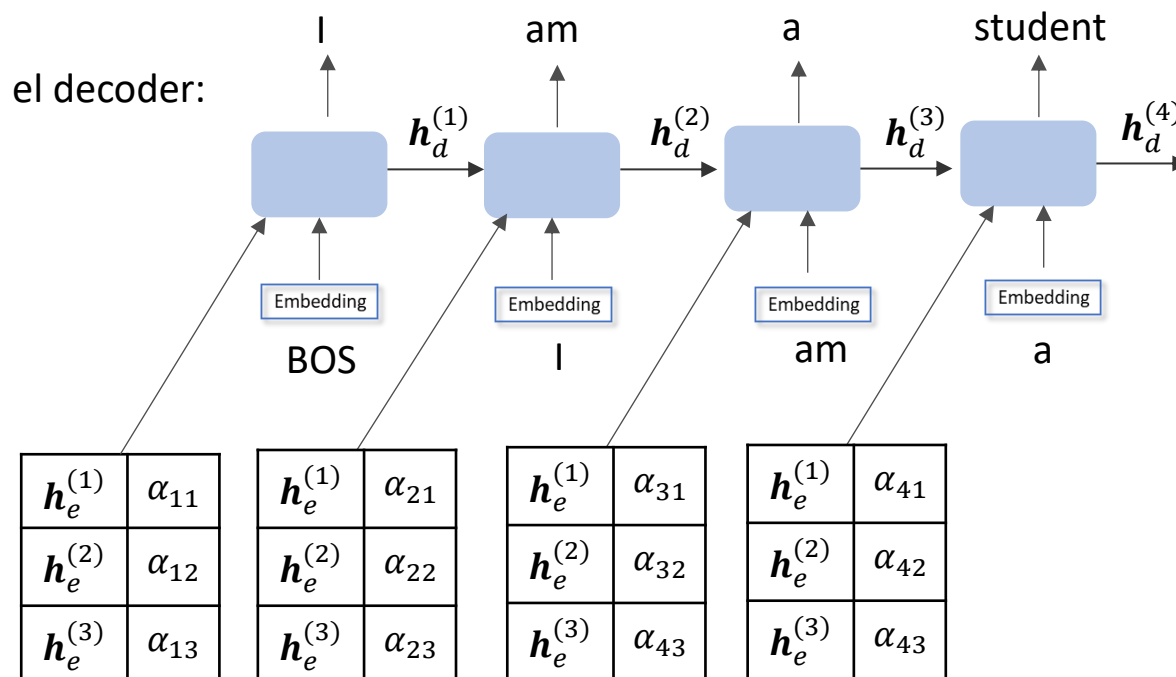
Asignar diferentes niveles de importancia a los elementos (estados), en cada paso de procesamiento = **ponderar**

El mecanismo se basa entonces en asignar **pesos de atención** a cada elemento de la secuencia, en cada paso de tiempo.

En el encoder:



En el decoder:

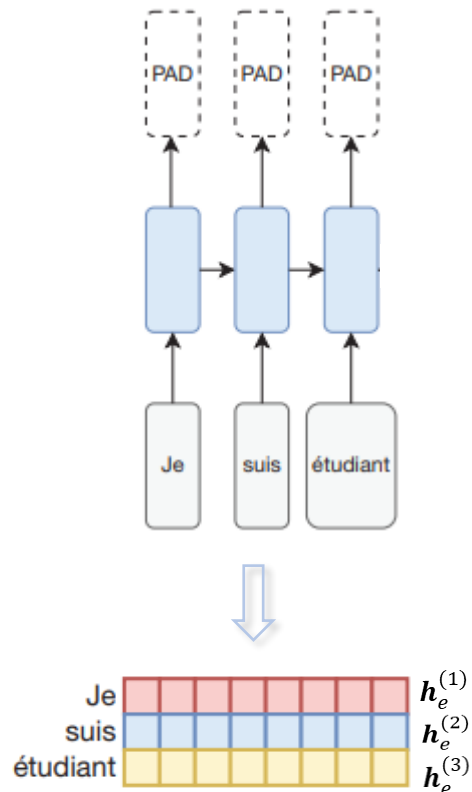


α_{tj} = peso en el tiempo t del decoder para el estado i del encoder

¿Cómo modificar el encoder y el decoder para implementar un mecanismo de atención?

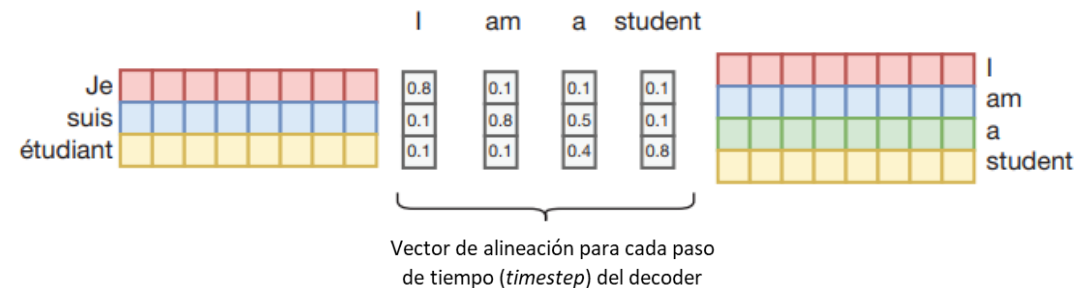
Encoder

Almacena el estado interno en cada paso de tiempo.

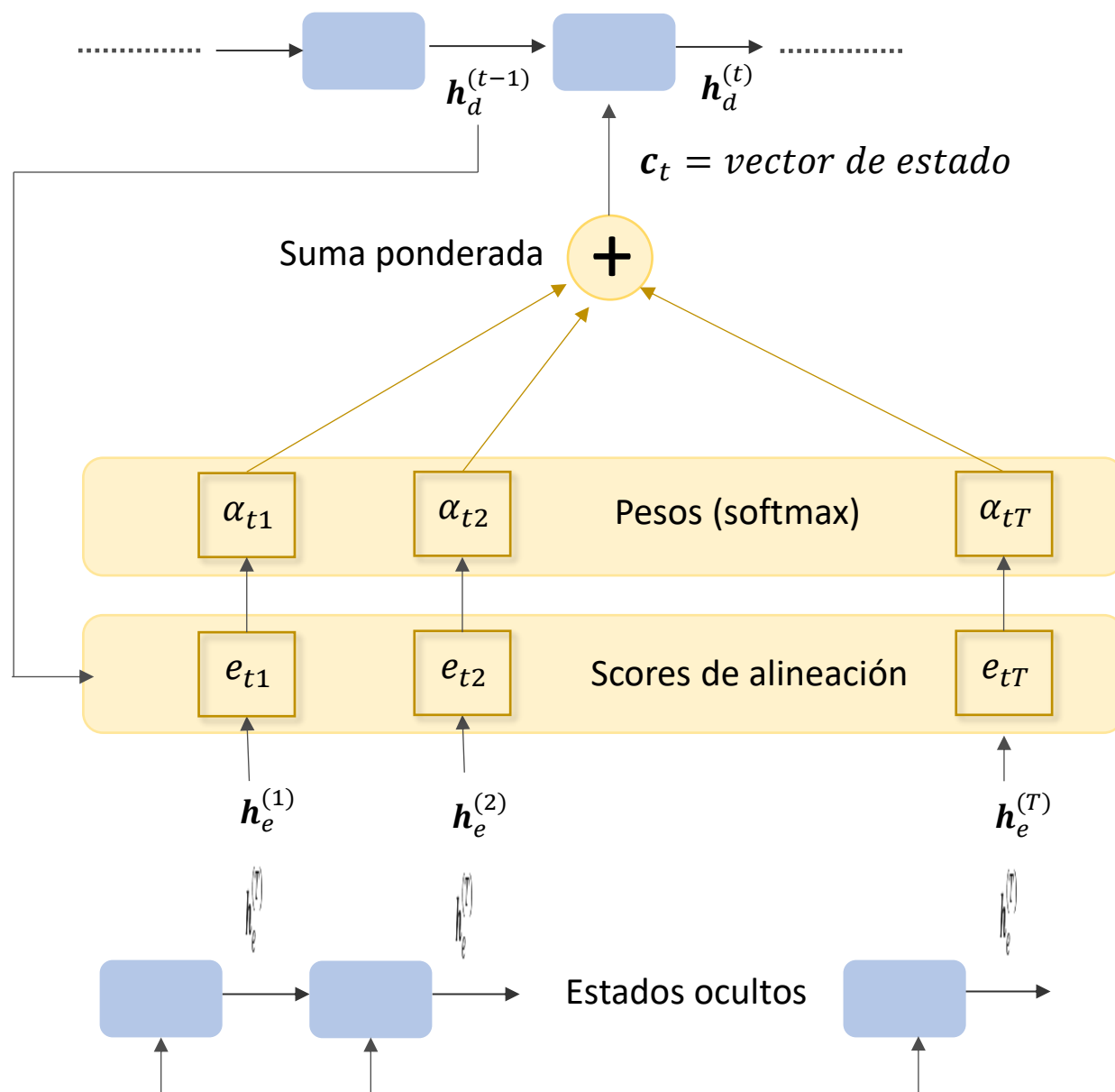


Decoder.

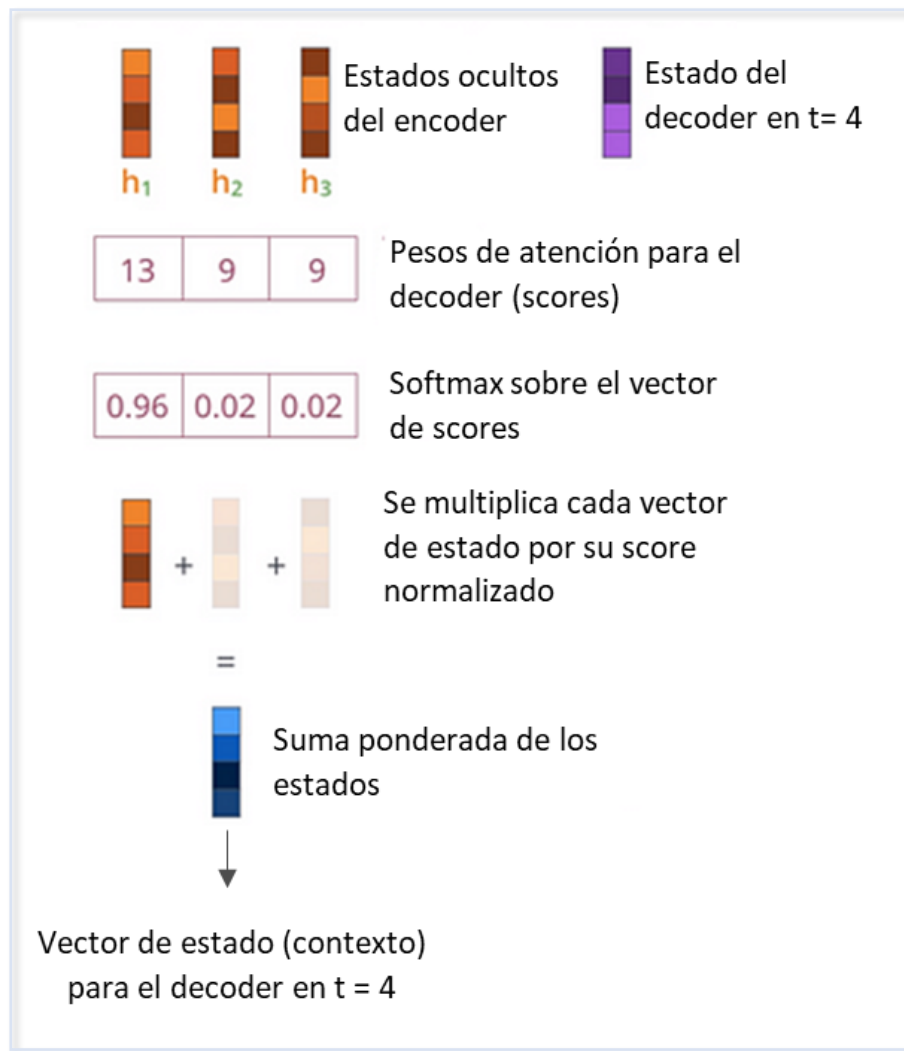
- Calcula un score de alineación (*alignment score*) para cada vector de estado (determina cuánta atención poner a cada vector de estado).
- Normaliza los scores de alineación con una función softmax y se construye el vector de alineación (*alignment vector*).
- Multiplica cada vector de estados por su score de alineación normalizado y suma los vectores resultantes (suma ponderada).
- Este vector de estado final (resultado de la suma ponderada) se utiliza como entrada al decoder durante el paso de tiempo actual, junto con el dato de entrada (se concatenan).



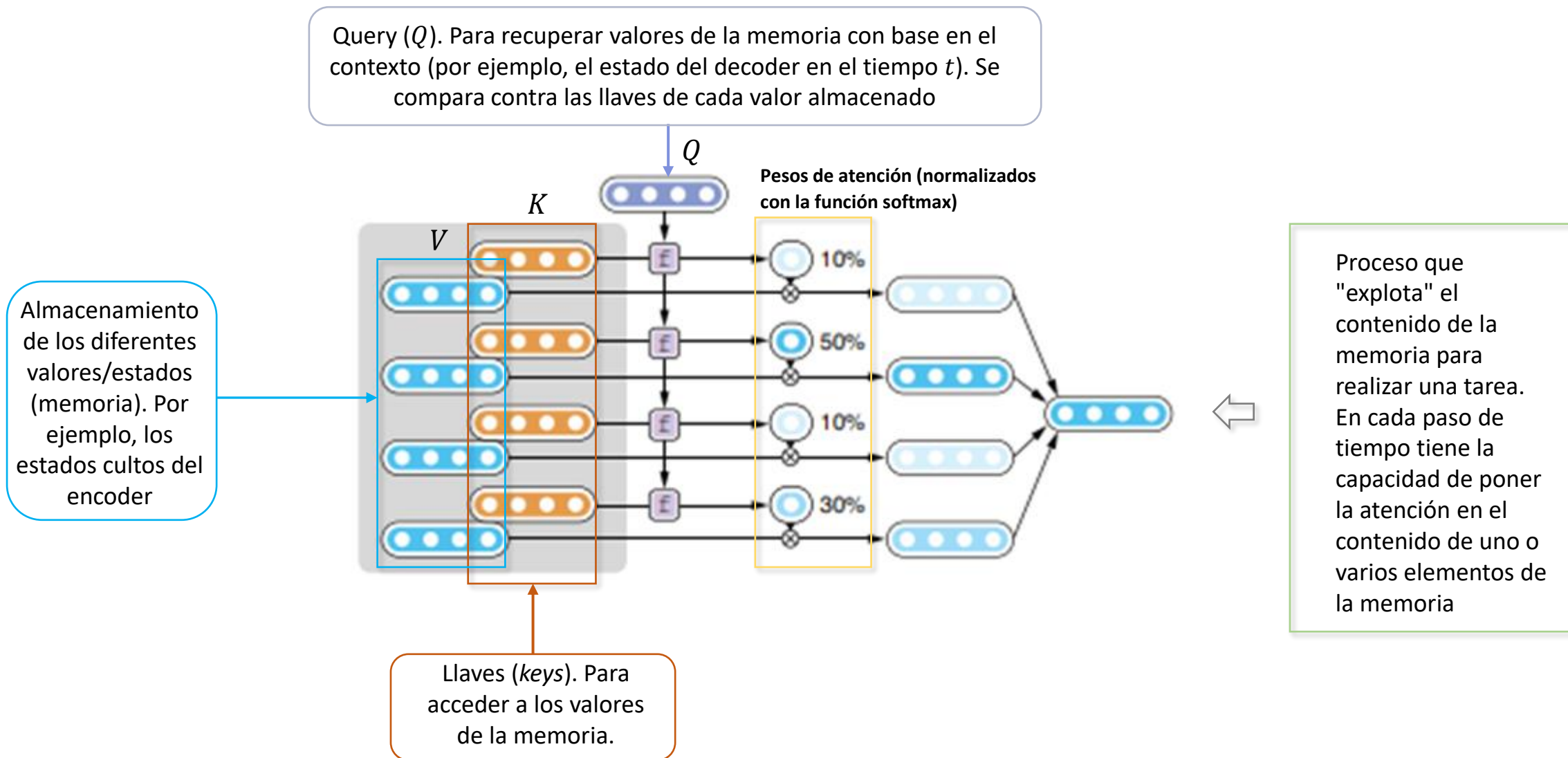
- Los pesos de atención son calculados en cada paso de tiempo del decoder (*timestep*) y están controlados por el estado interno del decoder del paso anterior. Este representa el contexto.
- Resultado: alineación de palabras de las dos secuencias (entrada y salida).

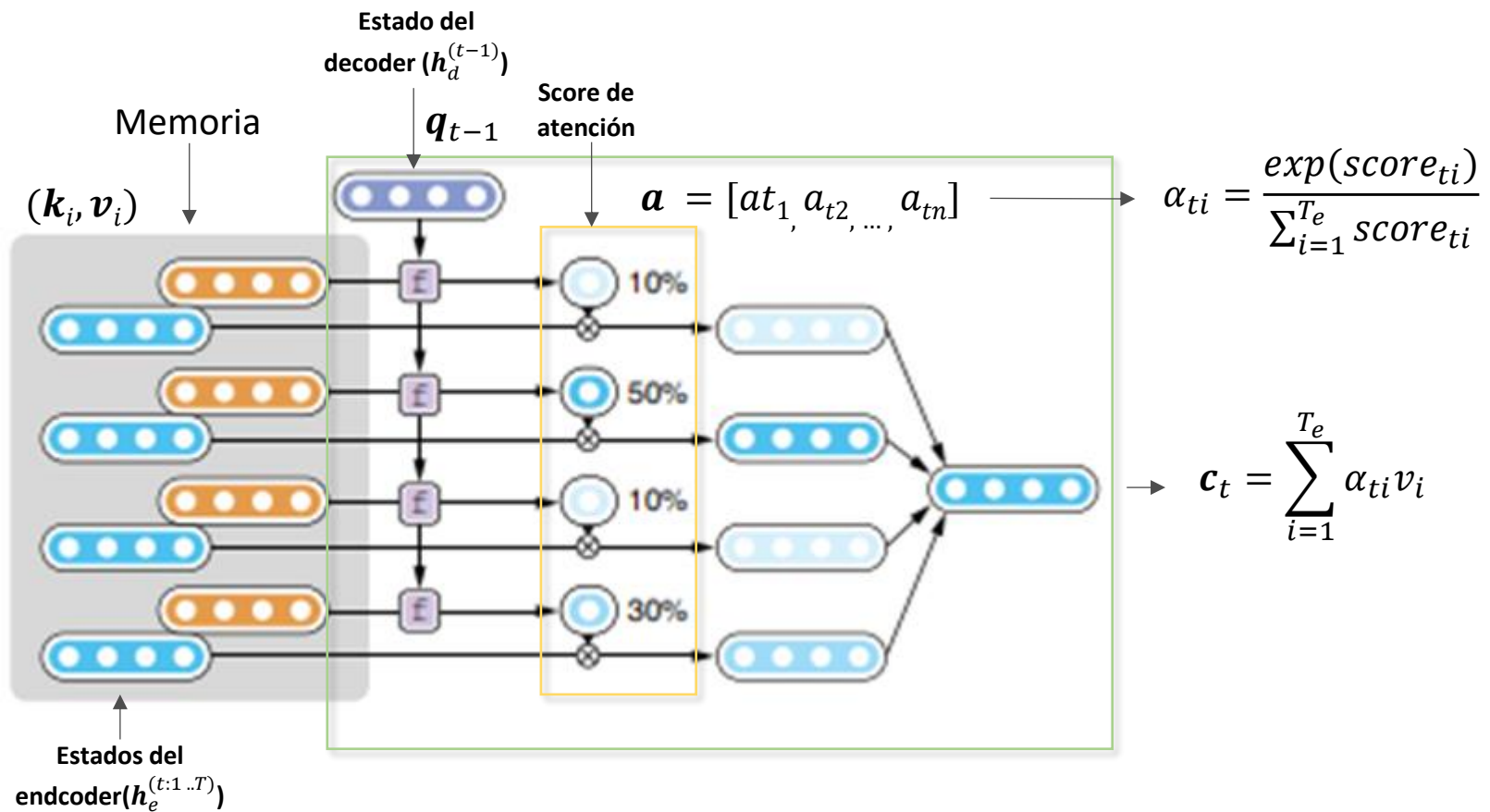


Ejemplo:



❑ En general, en un sistema de atención para ML:





Se puede expresar la atención con la siguiente fórmula:

$$Atención = softmax(QK^T)V$$

¿Cómo se mide la similitud?



$score(q_{t-1}, k_i) = q_{t-1} \cdot k_i$ Mecanismo de atención dot (Luong attention).

$score(q_{t-1}, k_i) = \frac{q_{t-1} \cdot k_i}{\sqrt{dim_k}}$ Mecanismo de atención dot escalado.

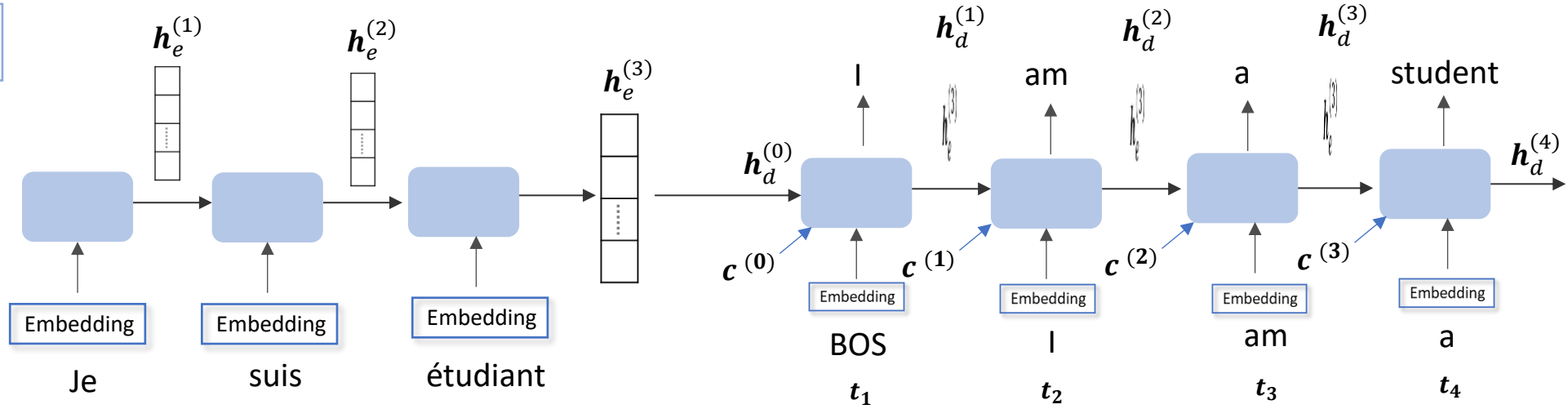
$score(q_{t-1}, k_i) = V_a(tan h(W_a(q_{t-1}, k_i)))$ Mecanismo de atención aditivo.

¿Qué hay en Keras?

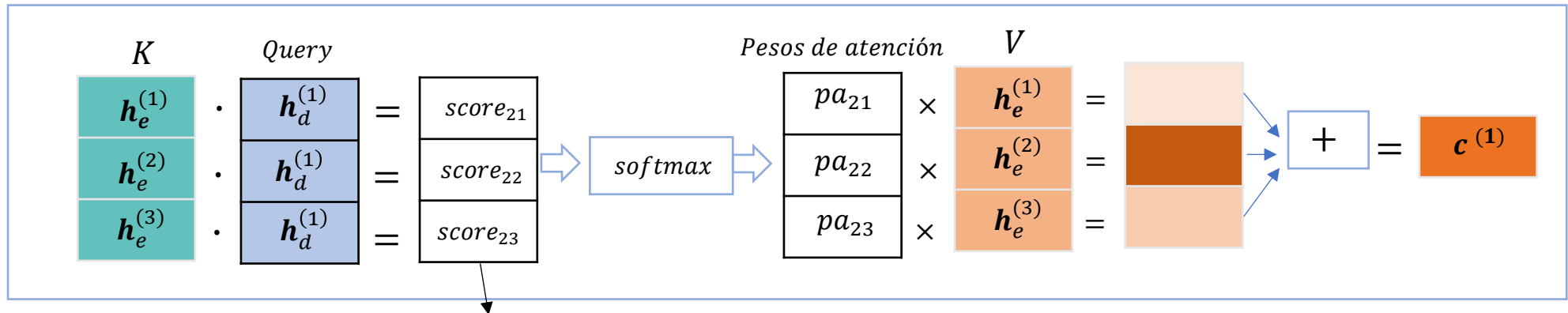
❏ En resumen...

$$a^{(t=i)} = h_e^{(i)}$$

$$a^{(t=i)} = h_d^{(i)}$$

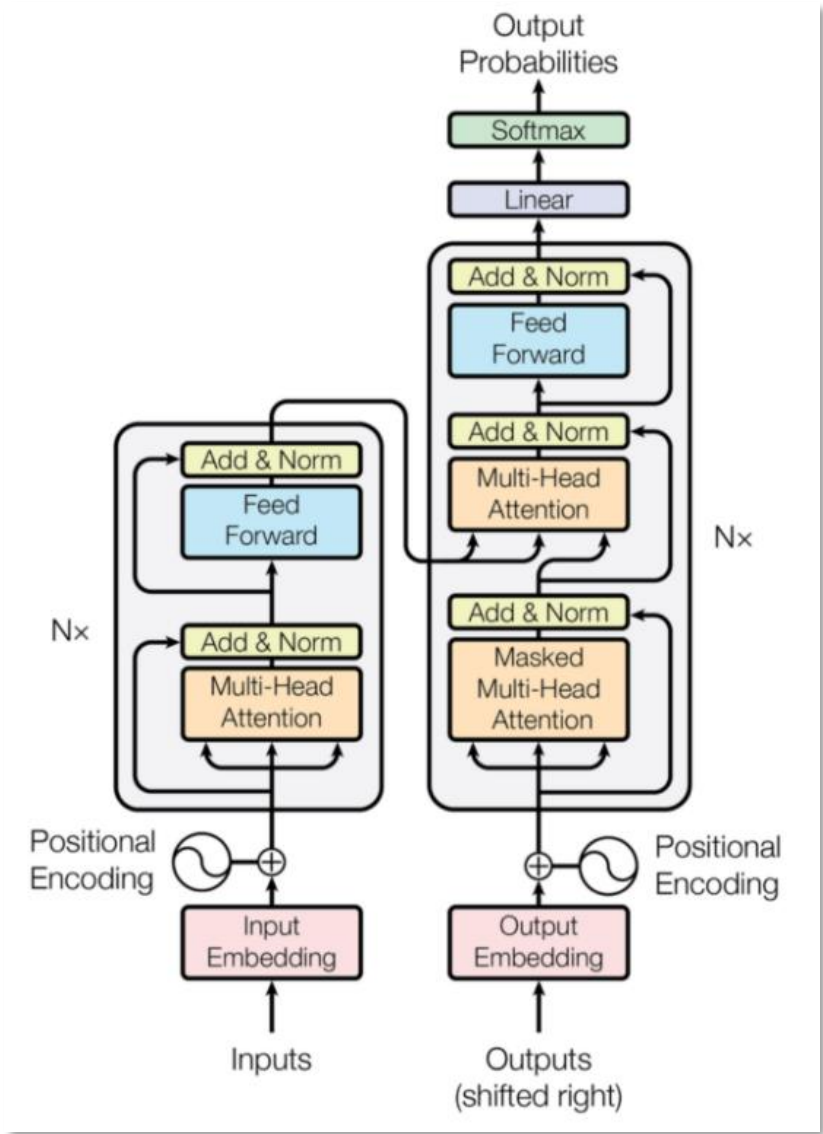


Ejemplo. En el tiempo t_2 del decoder:

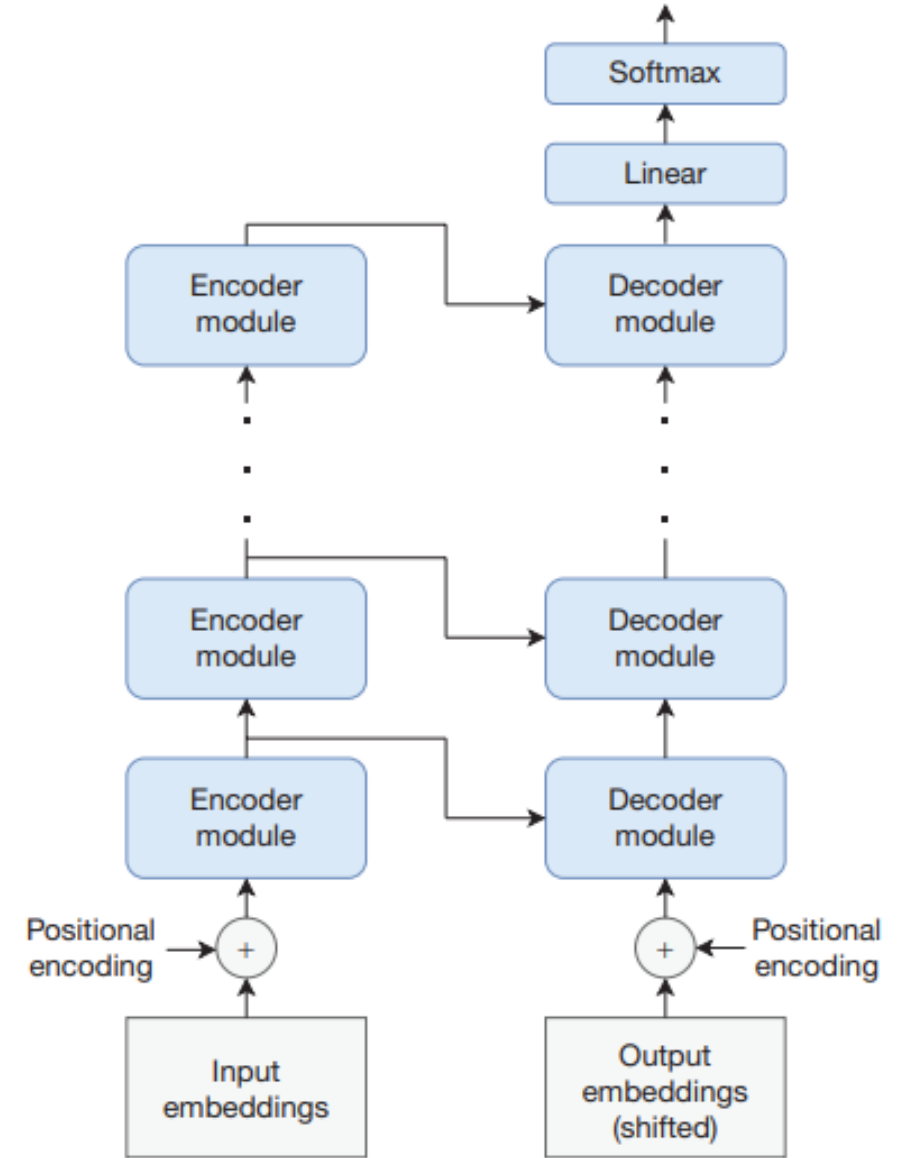


Los scores de alineaci3n miden la “similitud” entre el vector de estado del decoder en el paso de tiempo $t - 1$ y los estados ocultos del encoder.

Arquitectura Transformer



- Mecanismo de auto-atención (*self attention*) y atención multi-cabeza (*multi-head attention*).
- Codificación posicional de la entrada.



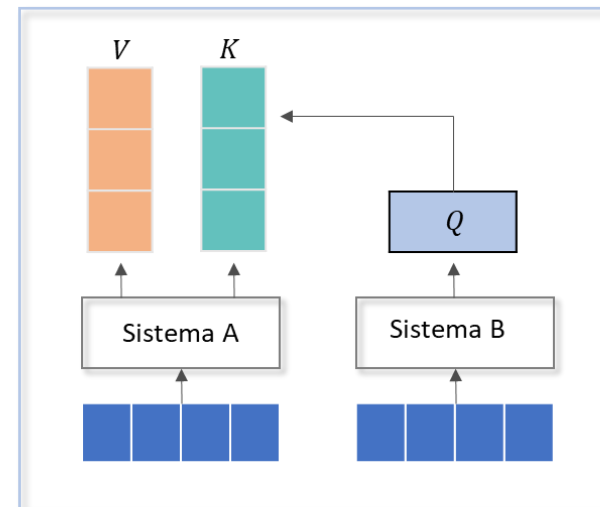
Auto-atención (*self-attention*)

En el mecanismo
de atención

La salida es una suma ponderada de la entrada,
dependiente del contexto.

El **query** = un estado del decoder.

Idea: comparar el estado del decoder en un
tiempo t contra todos los estados de encoder.



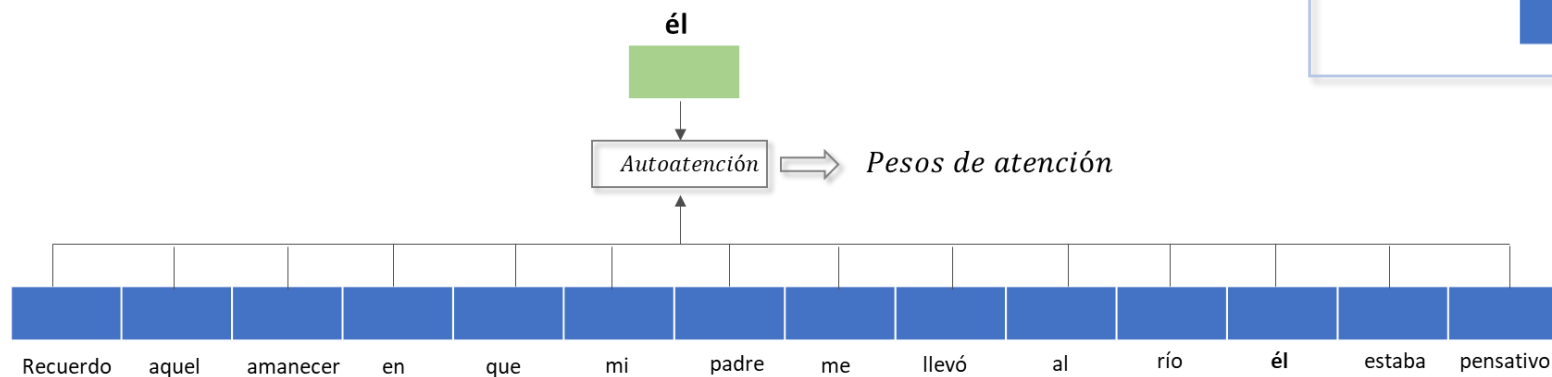
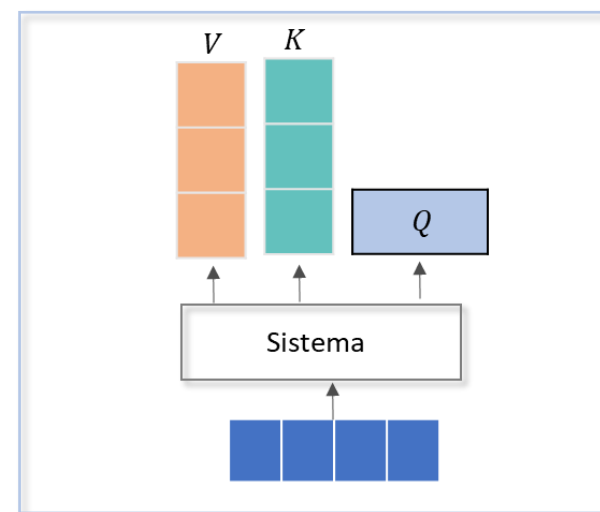
Atención
cruzada
(*cross
attention*)

En auto atención

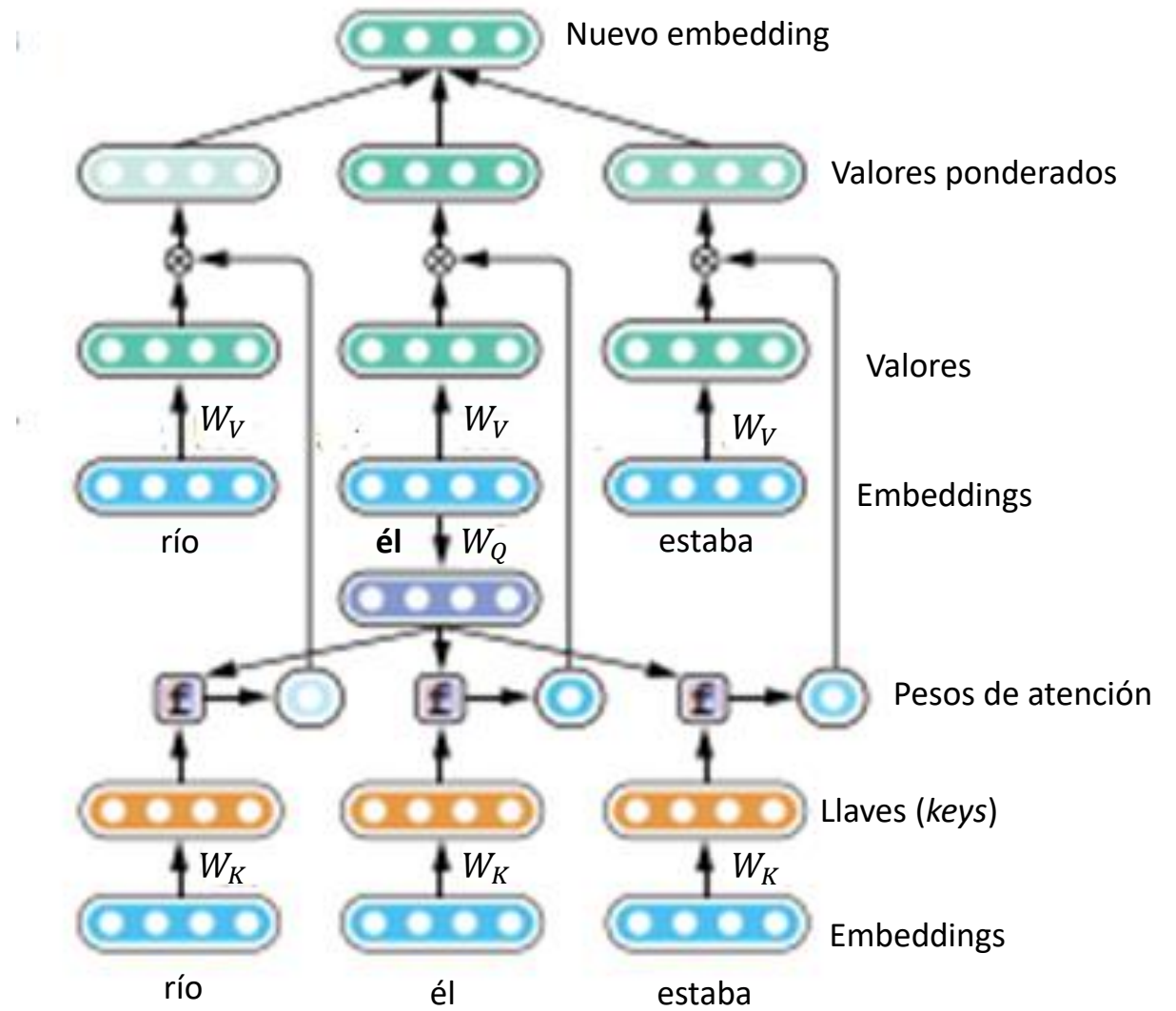
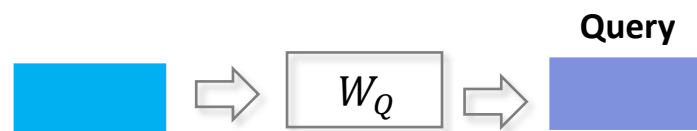
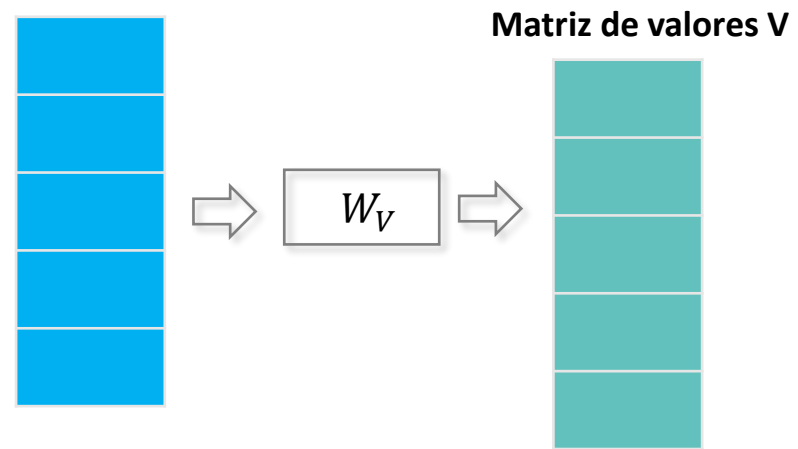
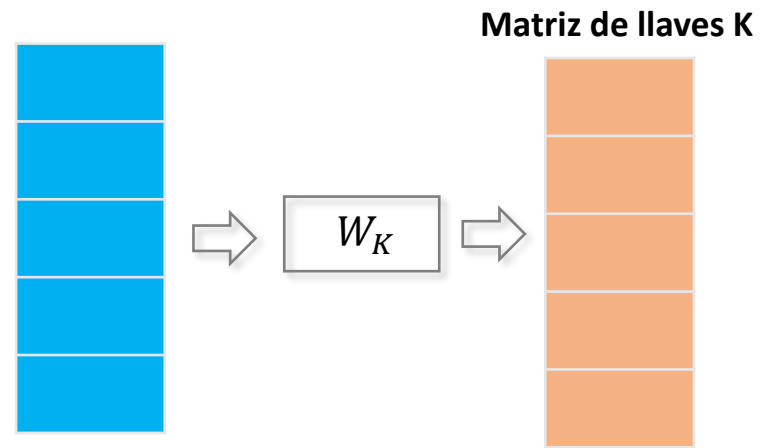
La salida es una suma ponderada de la entrada,
dependiente del contexto.

El **query** = un valor de la entrada

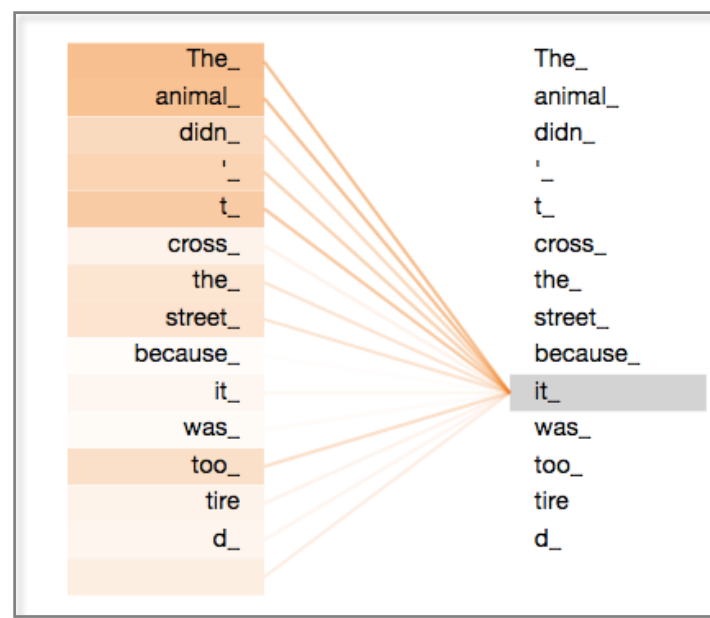
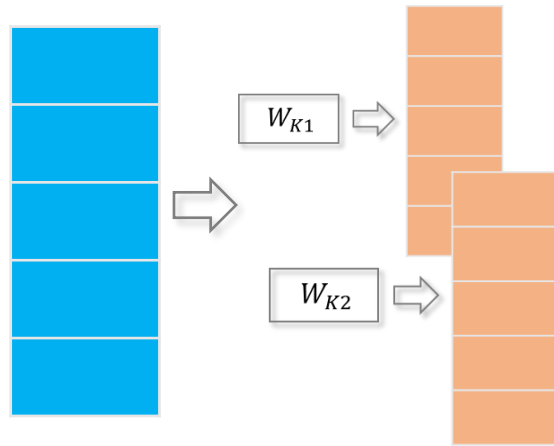
Idea: comparar un token de entrada (una
palabra) contra toda la secuencia de entrada
(todos los tokens de entrada)



¿Cómo se determinan V , K , y Q ?



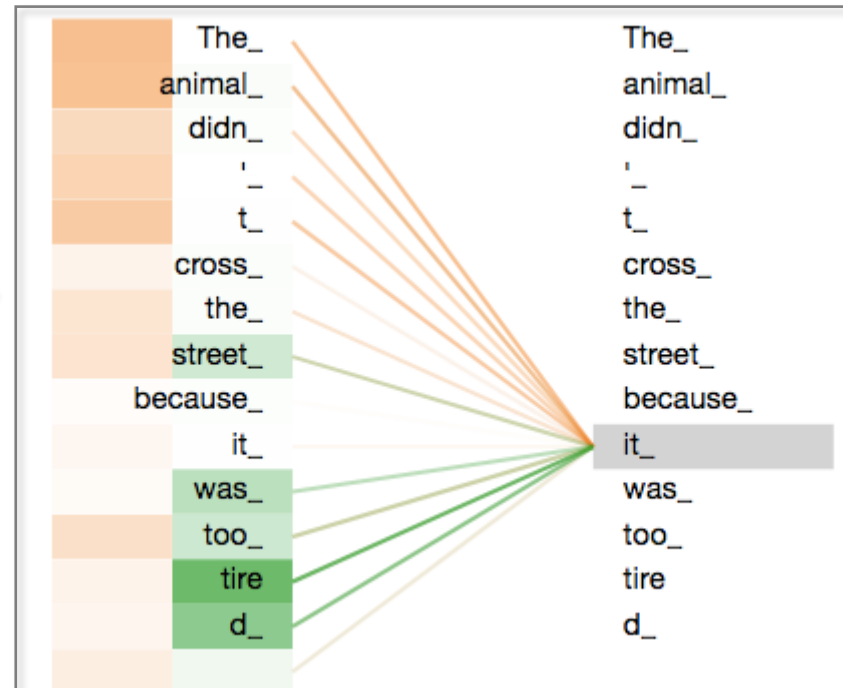
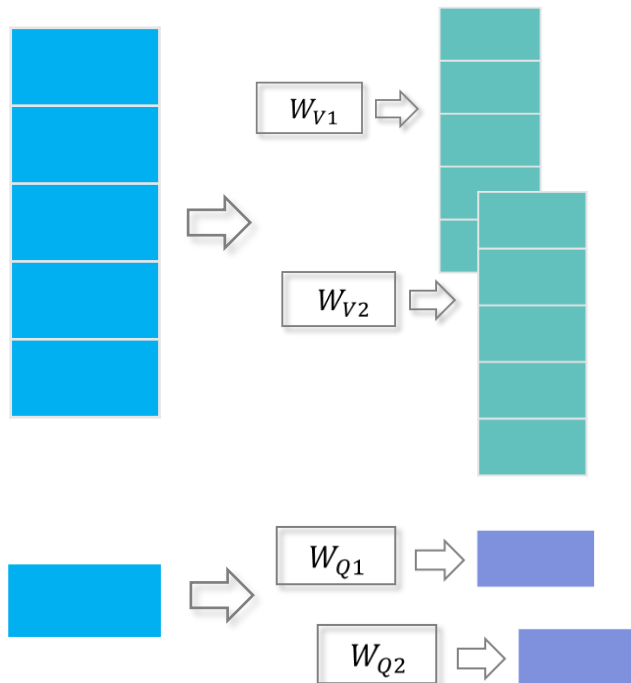
¿Y si se realiza varias veces?



Con un solo mecanismo de auto atención

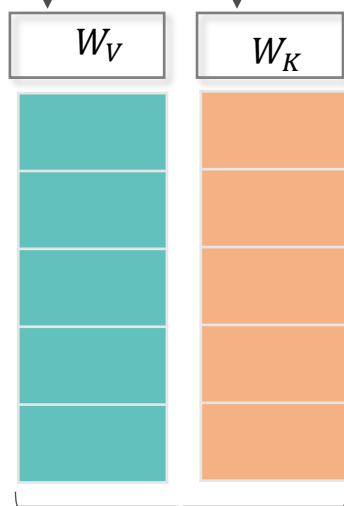
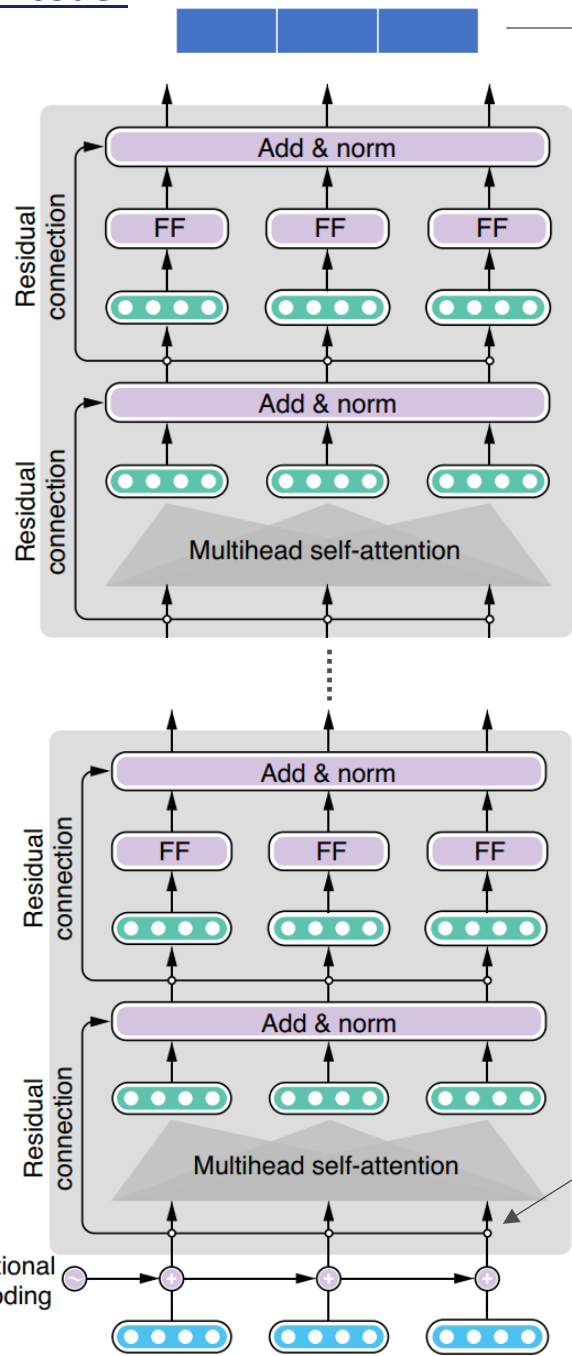
**Atención multi-cabeza
(multi-head attention)**

Con dos o más mecanismos de auto atención

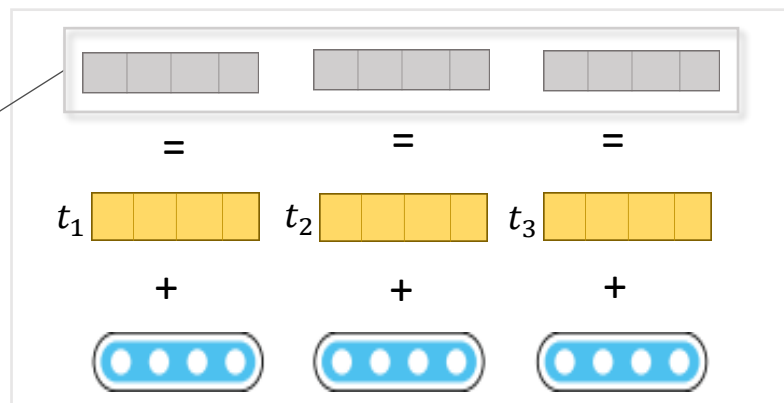


- Se utilizan múltiples conjuntos de valores, llaves y queries por token.
- Esto genera un conjunto de pesos de atención que permiten enfocarse en diferentes aspectos de la entrada.
- El embedding final es una combinación de los embeddings de cada “cabeza”

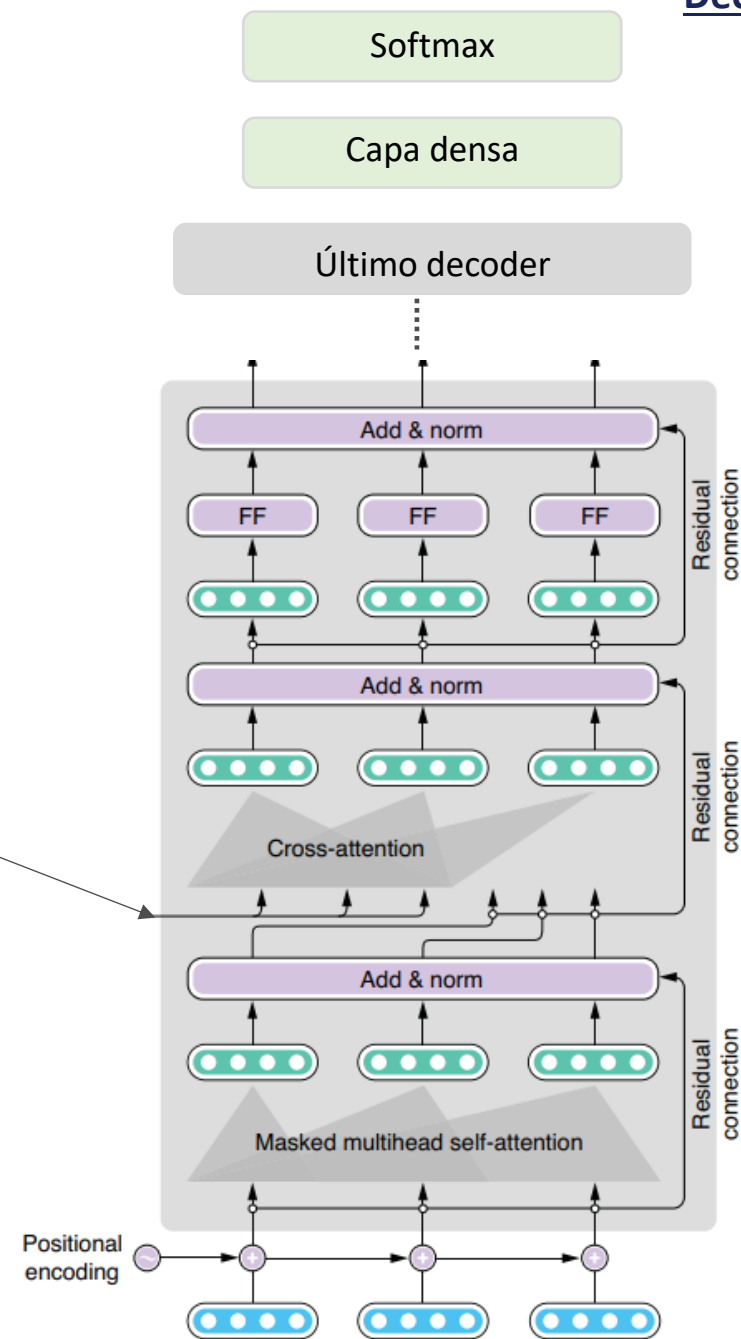
Encoder

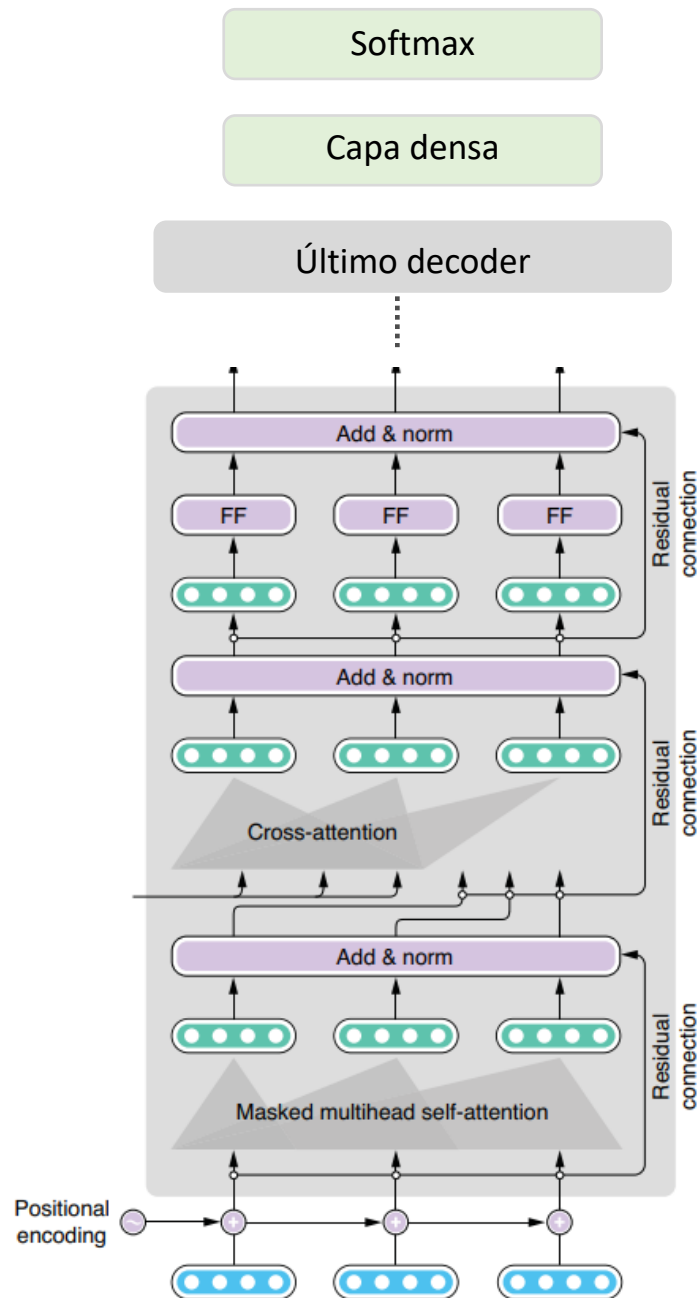


La salida del último codificador se transforma en un conjunto de vectores K y V . Cada decodificador los utilizará en su capa de "cross attention". Este mecanismo es similar al utilizado en el modelo seq2seq con atención.



Decoder





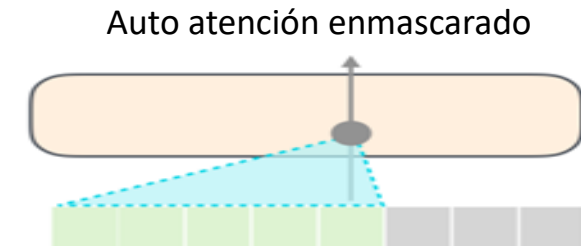
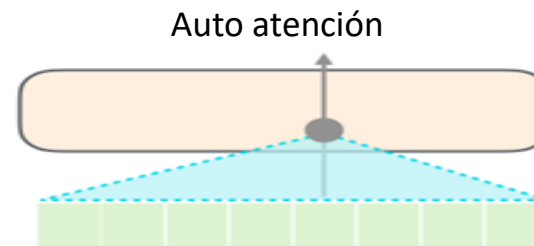
- Se utiliza el producto interno para determinar los scores de alineación, pero escalados a la dimensión del vector de entrada.

$$\text{Atención} = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

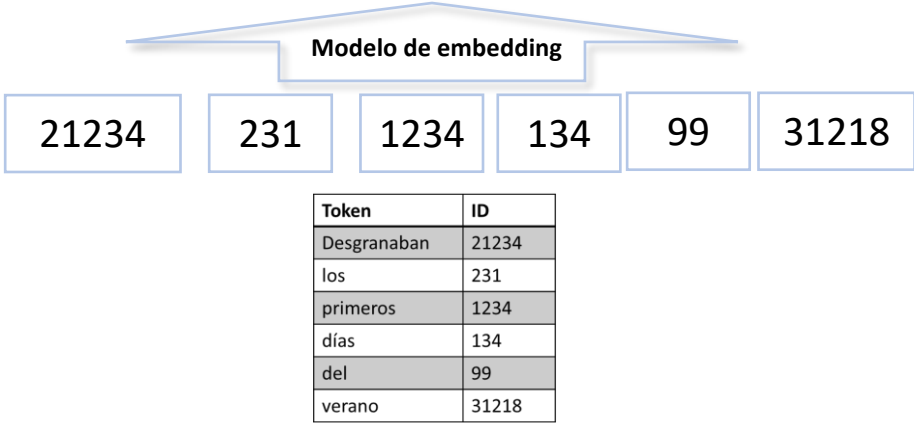
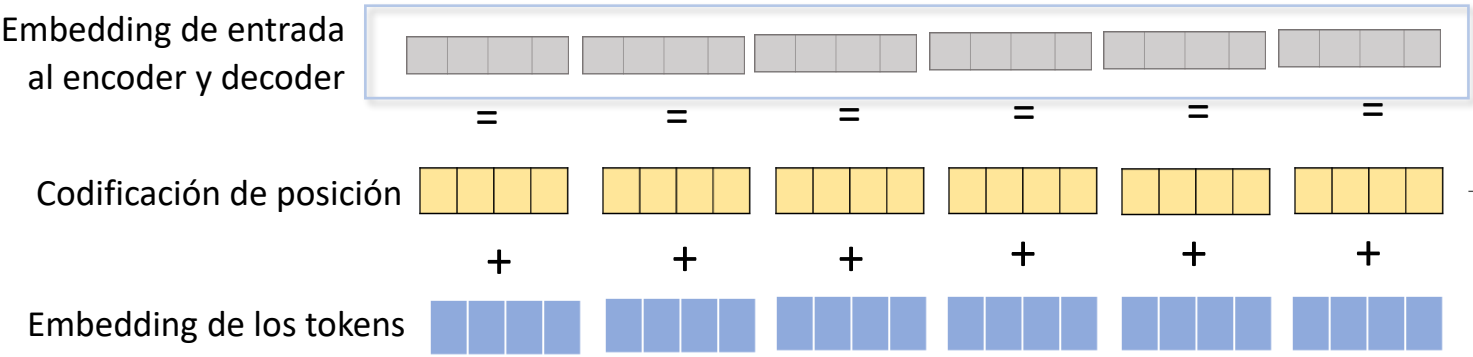
- Tamaño de la capa de salida (softmax) = vocabulario utilizado para el lenguaje destino.
- Función de costo: *cross entropy*.
- La codificación posicional en el paper original utiliza funciones seno y coseno de diferentes frecuencias:

$$PE_{(pos,i)} = \begin{cases} \sin\left(\frac{pos}{10000^{i/d_{\text{model}}}}\right) & \text{if } i \bmod 2 = 0 \\ \cos\left(\frac{pos}{10000^{(i-1)/d_{\text{model}}}}\right) & \text{otherwise} \end{cases}$$

- Aunque el encoder no procesa la entrada de manera secuencial, el modelo completo sí es autorregresivo (el decoder produce un token de salida en cada paso de tiempo).
- Para ello utiliza un mecanismo de atención “enmascarado”:



Con respecto a la codificación posicional:



Tokens Desgranaban los primeros días del verano

Tokenización

Entrada Desgranaban los primeros días del verano

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

pos	posición del token en la secuencia (0, 1, 2, ...)
dmodel	Tamaño del vector de embedding
i	Índice de la pareja de dimensiones seno-coseno dentro del vector. Cada valor define una frecuencia espacial distinta.

Por ejemplo, para “primeros” pos = 2 y se tendría:

0.91 -0.42 0.02 1.00

Modelos de lenguaje

¿Qué es un modelo de lenguaje?

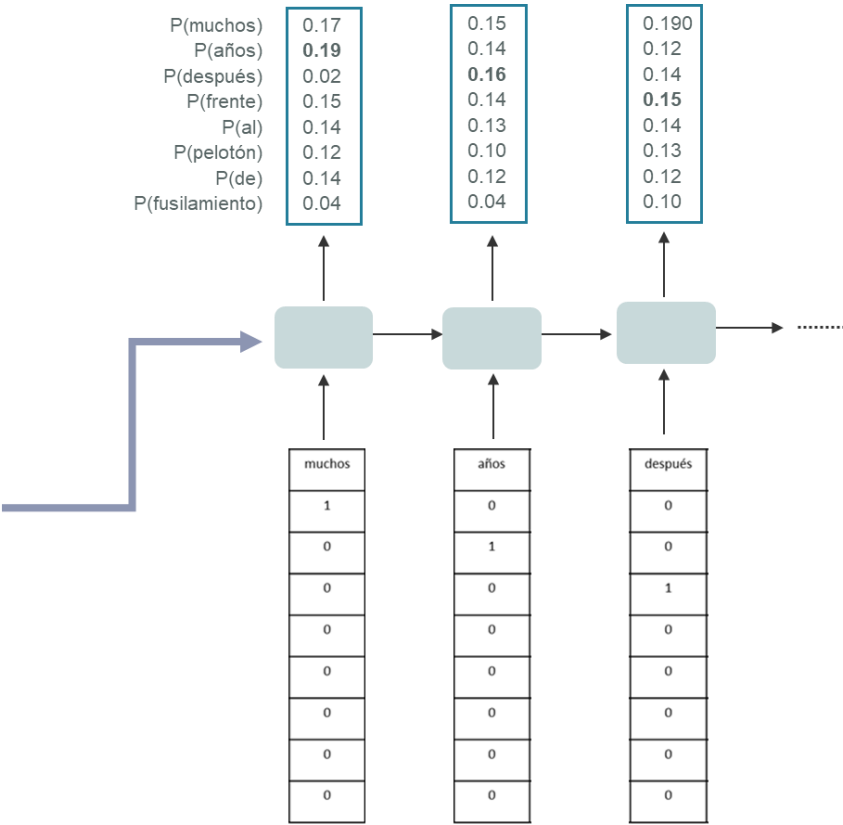
- **Permiten identificar la verosimilitud de las palabras en una frase** (Kumar, A., Karthika, D. (2023). Deep Learning Approach for Natural Language Processing, Speech, and Computer Vision. Techniques and Use Cases. CRC Press).
- **Los modelos lingüísticos predicen la palabra siguiente a partir de las palabras anteriores del texto** (Paaß, G., Giesselbach, S. (2023). Foundation Models for Natural Language Processing. Pre-trained Language Models Integrating Media. Springer).



Los modelos de lenguaje (o lingüísticos) construyen una distribución de probabilidad sobre palabras o secuencia de palabras. Estas probabilidades son utilizadas para predecir la siguiente palabra más probable en una frase con base en la secuencia dada.

“Muchos años después, frente al pelotón de fusilamiento”

	muchos	años	después	frente	al	pelotón	de	fusilamiento
muchos	1	0	0	0	0	0	0	0
años	0	1	0	0	0	0	0	0
después	0	0	1	0	0	0	0	0
frente	0	0	0	1	0	0	0	0
al	0	0	0	0	1	0	0	0
pelotón	0	0	0	0	0	1	0	0
de	0	0	0	0	0	0	1	0
fusilamiento	0	0	0	0	0	0	0	1



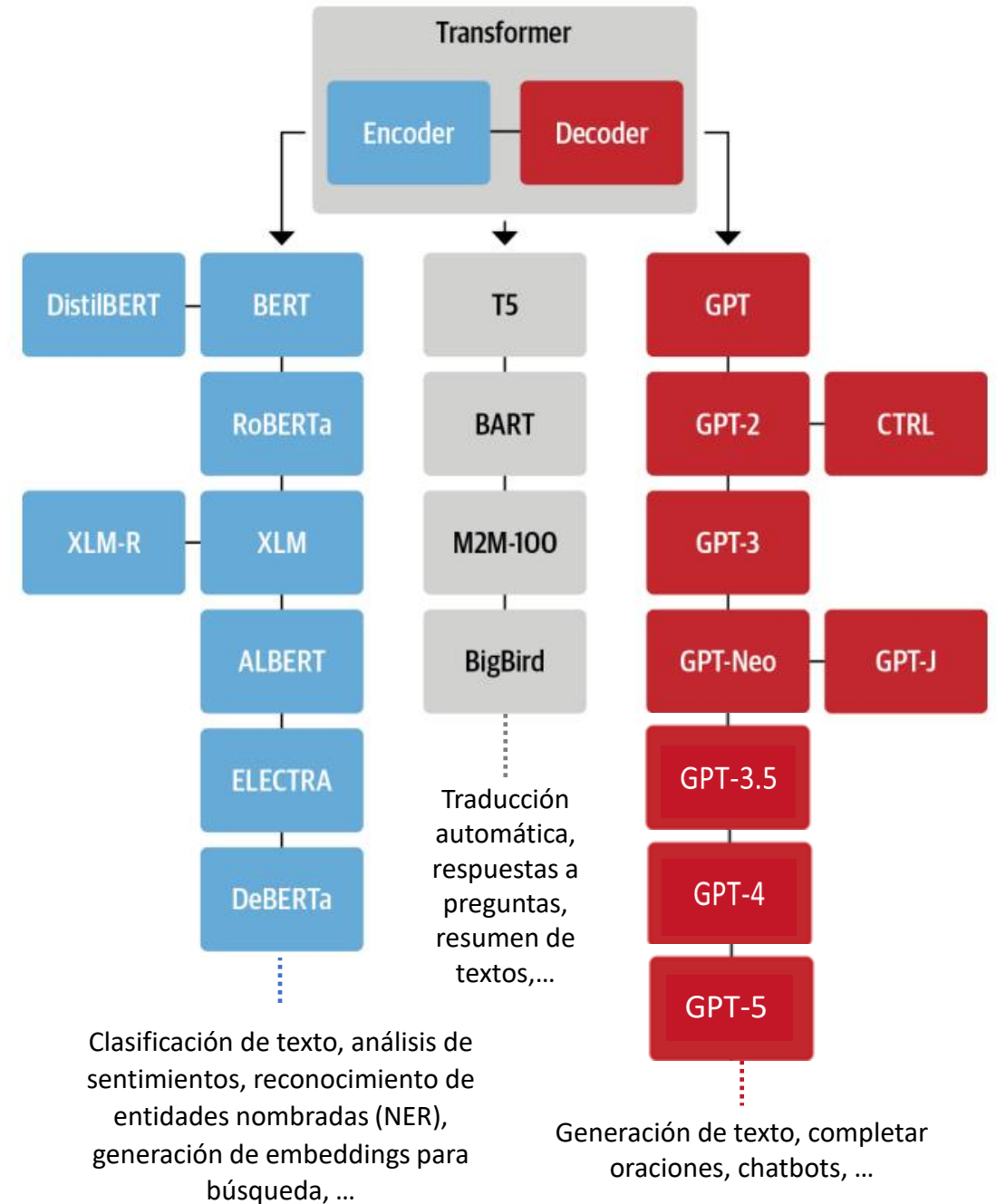
Con Transformers:

- En procesamiento del lenguaje natural: clasificación de textos, reconocimiento de entidades nombradas, respuesta a preguntas, **modelado del lenguaje**, resumen, traducción, selección múltiple y generación de textos.



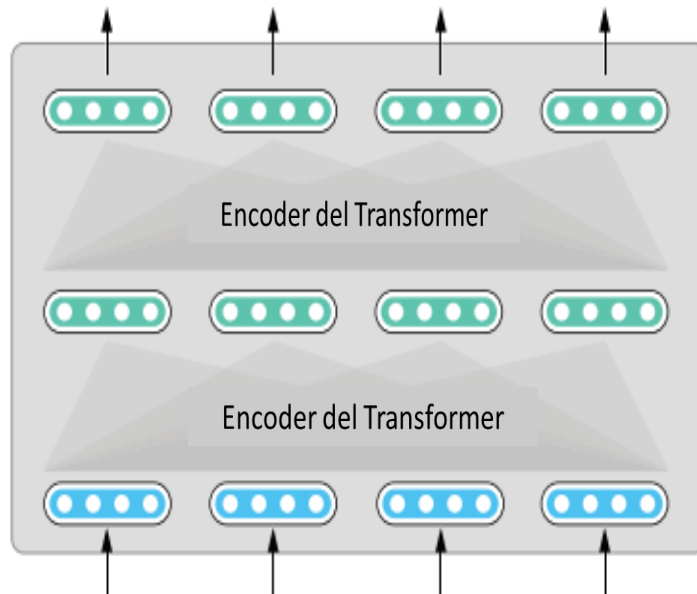
Otras aplicaciones:

- En visión artificial: clasificación de imágenes, detección de objetos y segmentación.
- En procesamiento de audio: reconocimiento automático del habla y clasificación de audio.
- En procesamiento multimodal: descripción de imágenes, generar imágenes a partir de texto, reconocimiento óptico de caracteres, clasificación de vídeos y respuesta a preguntas visuales.



❏ El modelo BERT

- BERT (*Bidirectional Encoder Representations from Transformers*) es un modelo de lenguaje diseñado para tareas de comprensión del lenguaje natural.
- Se centra en representar palabras con base en el contexto $\Rightarrow$ construye **embeddings contextuales**.



- Puede tomar el contexto en ambas direcciones.
- Disponible en diferentes tamaños: base (12 encoders, 110 millones de parámetros) y largo (24 encoders, 345 millones de parámetros).

Limitaciones de los embeddings estáticos o no contextuales (Ej: word2vec):

“El banco no abrió hasta las nueve de la mañana. Mientras, estuve esperando sentado en un banco viendo a la gente pasar.

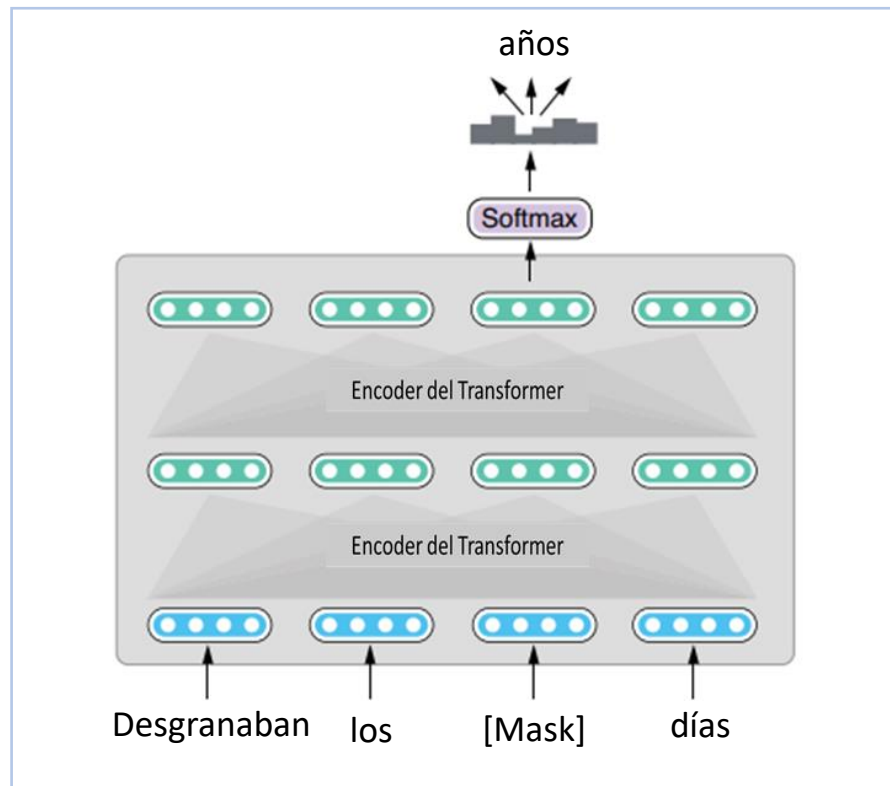
¡Solo hay una representación vectorial para la palabra “banco”!

Pero, las palabras pueden tener más de un significado dependiendo del contexto.

Idea: que las palabras tengan diferentes **representaciones** dependiendo del contexto.

¿Cómo se entrena?

- Utiliza aprendizaje auto supervisado (*self supervised learning*)
- Tener en cuenta que BERT = **encoder** del Transformer $\Rightarrow$ transforma la entrada en una serie de embeddings que tienen en cuenta el contexto.
- Además, para preentrenar embeddings $\Rightarrow$ predecir las palabras circundantes con base en la palabra objetivo.
- Para el preentrenamiento de modelos lingüísticos unidireccionales esta estrategia es adecuada. En el caso de los modelos bidireccionales como BERT ¿Qué hacer?
- $\Rightarrow$ Modelo de lenguaje enmascarado (MLM) = enmascarar palabras al azar en una frase determinada y entrenar para que el modelo prediga cuál es la palabra “eliminada”.

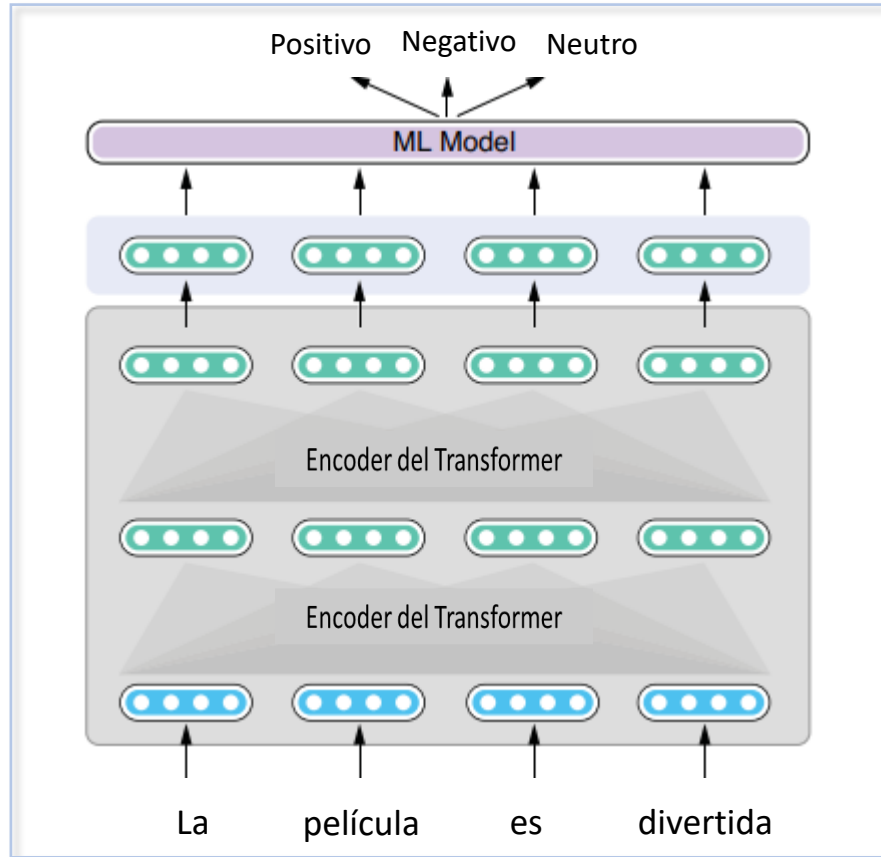


- Se sustituyen un pequeño porcentaje de palabras de una frase por un marcador de posición especial, como [Mask].
 - Después del último encoder hay una capa densa. Luego, la capa de salida, que es una softmax para derivar una distribución de probabilidad sobre las posibles palabras que pueden rellenar ese espacio en blanco.
 - Función de costo = entropía cruzada
-
- También utiliza otra estrategia de aprendizaje auto supervisado: predicción de la frase siguiente (*next-sentence prediction*, NSP): dadas dos frases, predecir si la segunda es la siguiente a la primera.

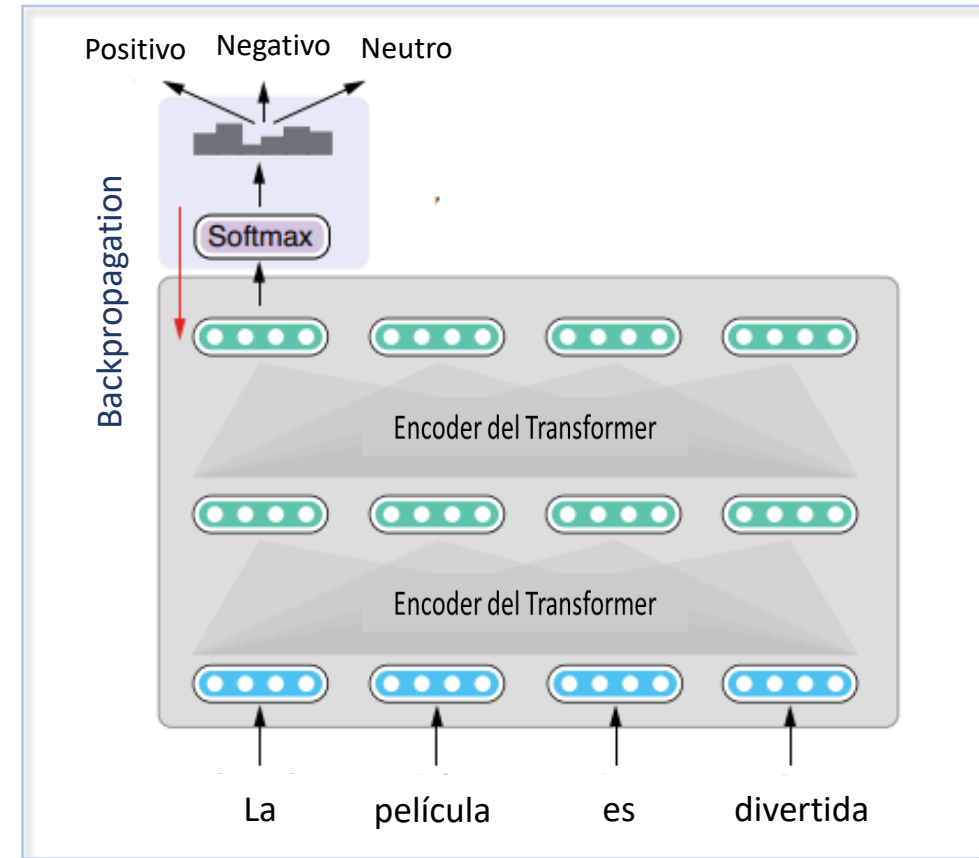
¿Cómo se usa?

- Mediante aprendizaje por transferencia:

Utilizar BERT como extractor de características



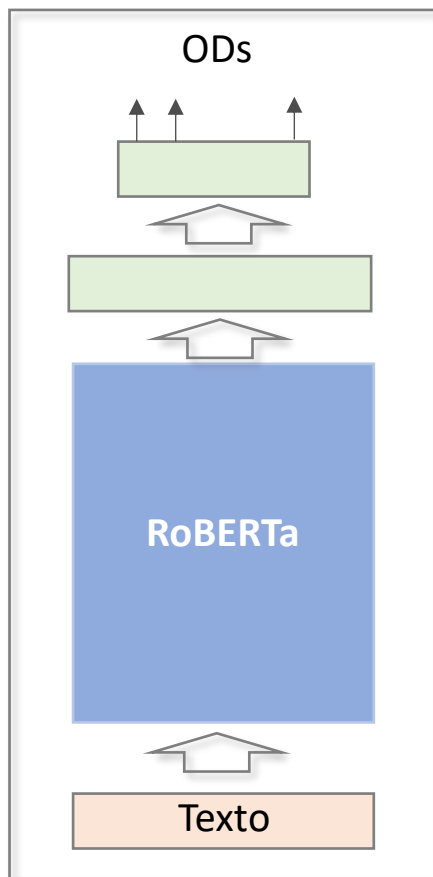
Mediante ajuste fino (*fine tuning*)



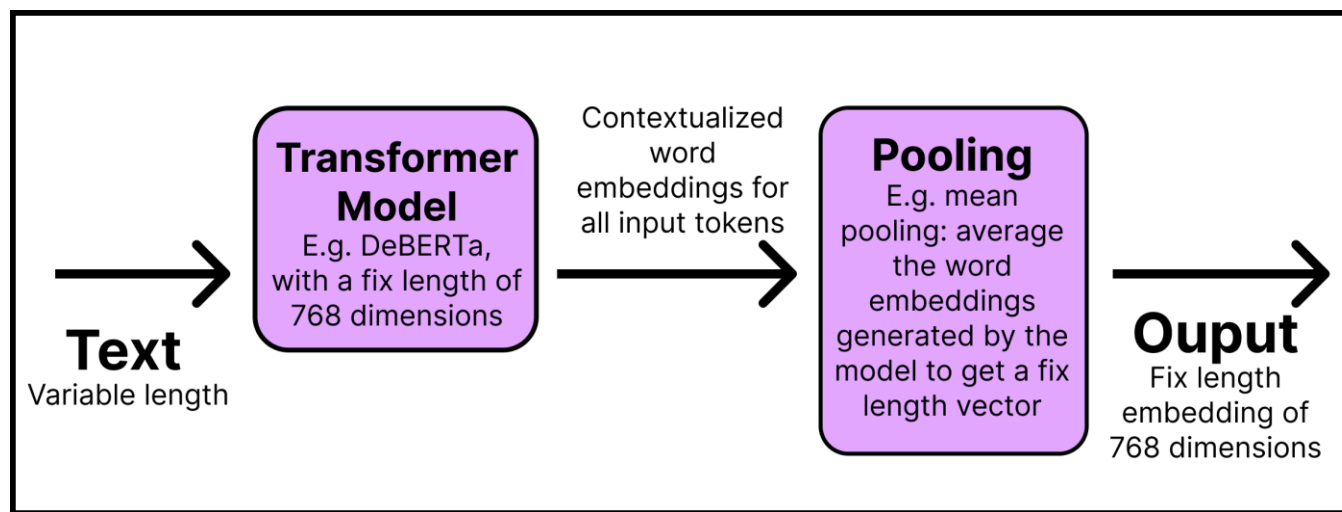
Importante: el tokenizador se encarga de generar los tokens (ID) así como los embeddings de entrada.

Ejemplo:

- Proyecto. Uso de la inteligencia artificial para apoyar el análisis de opiniones en procesos de planeación participativa y su traducción al lenguaje de los objetivos de desarrollo sostenible.
- Objetivo. Desarrollar un modelo que permita la clasificación de textos obtenidos mediante encuestas realizadas en territorios en los ODS planteados en la Agenda 2030 de las naciones unidas, haciendo uso de diferentes técnicas de aprendizaje profundo.



¿Es posible también generar Embeddings para una oración o documento?



<https://huggingface.co/sentence-transformers>

<https://sbert.net/>

Datos: OSDG Community Dataset.
<https://doi.org/10.5281/zenodo.7540165>

❑ El modelo GPT

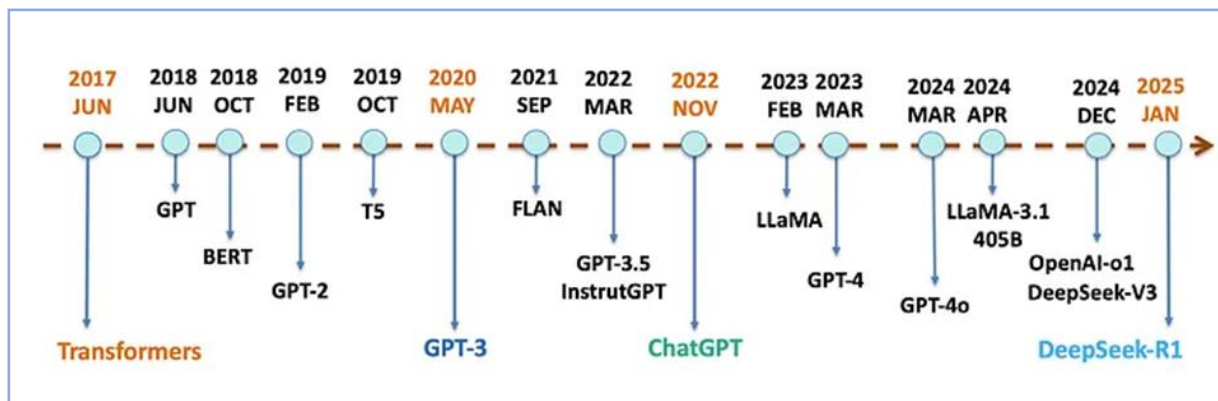
G: Generative $\Rightarrow$ Capaz de producir contenido

P: Pre-trained $\Rightarrow$ Entrenado con grandes conjuntos de datos.

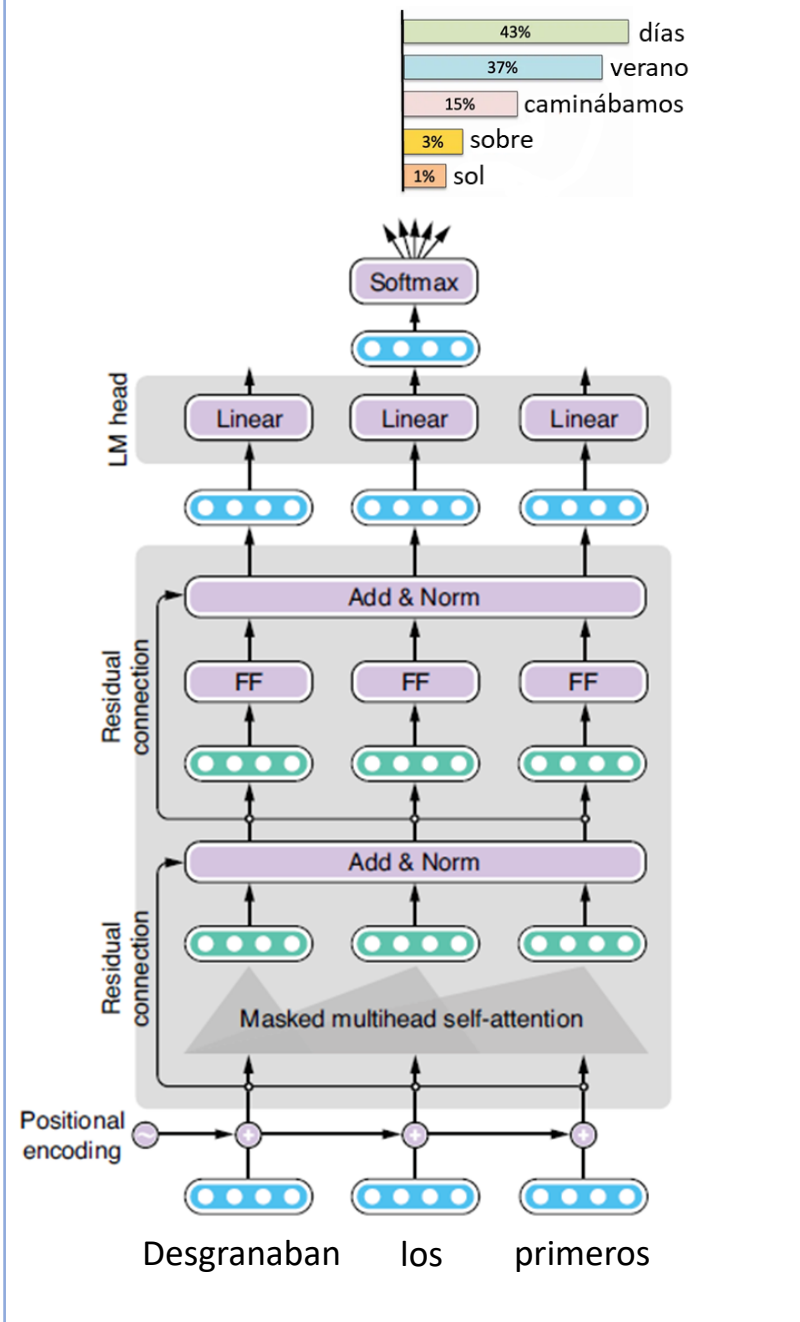
T: Transformer

- Tener en cuenta que GPT = **decoder** del Transformer

Los modelos de lenguaje (o lingüísticos) construyen una distribución de probabilidad sobre palabras o secuencia de palabras. Estas probabilidades son utilizadas para predecir la siguiente palabra más probable en una frase con base en la secuencia dada. Son modelos generativos



A Brief History of LLMs. From Transformers (2017) to DeepSeek-R1
(<https://medium.com/@lmpo/a-brief-history-of-lmms-from-transformers-2017-to-deepseek-r1-2025-dae75dd3f59a>)



Ejemplo:

Prompt: "**En un futuro cercano, la inteligencia artificial transformará la agricultura mediante**"

❑ Temperatura = 0.3, top_k = 10, top_p = 0.8

"**En un futuro cercano, la inteligencia artificial transformará la agricultura mediante** el uso de sensores, análisis de datos y sistemas automatizados que mejorarán la eficiencia y reducirán los costos de producción."

❑ Temperatura = 0.8, top_k = 50, top_p = 0.9

"**En un futuro cercano, la inteligencia artificial transformará la agricultura mediante** drones que supervisan los cultivos, algoritmos que anticipan sequías y plataformas que aprenden del suelo para optimizar cada semilla sembrada."

❑ Temperatura = 1.5, top_k = 100, top_p = 0.98

"**En un futuro cercano, la inteligencia artificial transformará la agricultura mediante** granjas parlantes, nubes que aconsejan a los agricultores y semillas que negocian su propio crecimiento con el clima."

¡Investigar sobre estos parámetros para la próxima clase!

Próxima sesión

- Entrenamiento y post entrenamiento de grandes modelos de lenguaje.